FACESYNC

AI Wellness Mirror

Complete Build Master Plan — From Scratch

Every Feature · Every Phase · Every File · Full Stack

Attribute	Detail
Project name	FaceSync — AI Wellness Mirror
Build type	Ground-up from scratch — no legacy code carried forward
Total phases	4 Phases over 9 months
Core stack	Python 3.11+ · React 18 + TypeScript · Node.js/Express BFF
AI stack	MediaPipe · TensorFlow/Keras · scikit-learn · InsightFace · SpeechBrain · Whisper
Data privacy	On-device only · SQLCipher · GDPR/CCPA compliant · Zero cloud processing
This document	Master build plan synthesising every feature discussed in this project

SCOPE OF THIS DOCUMENT

This master plan consolidates EVERY feature discussed across the entire project:

- AI wellness mirror (posture, fatigue, emotion, voice state, fusion)
- RAVDESS vocal state classifier (calm / stressed / fatigued)
- Multi-modal face + voice recognition with identity enrollment
- React + Node.js/Express frontend + BFF replacing Flask static UI
- On-device privacy-first architecture with SQLCipher encryption
- Online learning, personalised baselines, session trend dashboard
- GDPR/CCPA/BIPA-compliant data governance
- Complete CI/CD pipeline, testing strategy, and deployment plan

TABLE OF CONTENTS

TABLE OF CONTENTS ... 2
PART 1 — THE BIG PICTURE: WHAT YOU ARE BUILDING ... 4
1.1 System in One Sentence ... 4
1.2 Complete Feature List (Everything You Requested) ... 4
Core AI pipeline: ... 4
Identity layer (new): ... 4
Frontend & dashboard (full replacement of Flask static): ... 4
Privacy & security (full compliance stack): ... 4
Infrastructure & operations: ... 5
1.3 Technology Stack — Complete ... 5
PART 2 — COMPLETE PROJECT STRUCTURE ... 7
2.1 Full Monorepo Directory Tree ... 7
PART 3 — 4-PHASE BUILD PLAN ... 9
3.1 Phase Overview ... 9
Phase 1 — Week-by-Week ... 9
Week 1: Project Scaffold & Capture ... 9
Week 2: Face Detection & Landmarks ... 9
Week 3: Fatigue & Emotion ... 9
Week 4: Voice Module & RAVDESS Pipeline ... 10
Week 5: Fusion, Overlay & Flask Server ... 10
Week 6: Online Learning & Phase 1 Hardening ... 10
Phase 2 — Week-by-Week ... 10
Week 7: ArcFace Face Recognition ... 10
Week 8: ECAPA-TDNN Speaker ID ... 11
Week 9: Enrollment Wizard ... 11
Week 10: Identity Service & Personalised Baselines ... 11
Week 11: Phase 2 Hardening & Consent System ... 11
Phase 3 — Week-by-Week ... 11
Week 12: BFF Scaffold & WebSocket Proxy ... 11
Week 13: Design Tokens & UI Primitives ... 12
Week 14: Pipeline & Chart Components ... 12
Week 15: Hooks & State ... 12
Week 16: Dashboard Page & Enrollment Wizard ... 12
Week 17: Vite Build Integration & Phase 3 Hardening ... 13
Phase 4 — Week-by-Week ... 13
Week 18: Full Privacy & Security Audit ... 13
Week 19: Robustness & Edge Case Testing ... 13
Week 20: Performance Hardening ... 13
Week 21: CI/CD & Docker ... 13
Week 22: Documentation, Final QA & Launch ... 14
PART 4 — DATASETS & TRAINING DATA ... 15
4.1 Dataset Summary ... 15
4.2 RAVDESS Label Mapping (Critical Detail) ... 15

PART 5 — AI MODEL REFERENCE 17
5.1 Model Summary Table 17
5.2 Fusion Formula 17
PART 6 — PERFORMANCE TARGETS & BENCHMARKS 19
6.1 All Performance Targets 19
PART 7 — PRIVACY, SECURITY & COMPLIANCE 20
7.1 Privacy Architecture Principles 20
7.2 Compliance Controls per Regulation 20
PART 8 — COMPLETE TESTING PLAN 21
8.1 Testing Pyramid 21
8.2 Critical Test Cases 21
AI model accuracy tests: 21
Identity tests: 21
Privacy tests: 21
PART 9 — DEPLOYMENT & HARDWARE 22
9.1 Minimum & Recommended Hardware 22
9.2 Installation Steps 22
9.3 Update Strategy 22
PART 10 — COMPLETE BUILD CHECKLIST 23
10.1 Phase 1 Checklist — Core AI Engine 23
10.2 Phase 2 Checklist — Identity Layer 23
10.3 Phase 3 Checklist — React Frontend + Node BFF 23
10.4 Phase 4 Checklist — Privacy, Polish & Deploy 24

PART 1 — THE BIG PICTURE: WHAT YOU ARE BUILDING

1.1 System in One Sentence

FaceSync is a real-time, fully on-device AI wellness mirror that identifies you by face and voice, analyses your posture, eye fatigue, facial emotion, and vocal state simultaneously, fuses all signals into a single wellness score, and shows you personalised, privacy-respecting feedback on a modern web dashboard — all without sending a single byte of your data to any server.

1.2 Complete Feature List (Everything You Requested)

Core AI pipeline:

- Face detection using MediaPipe BlazeFace TFLite (478 facial landmarks)
- Head pose estimation with OpenCV solvePnP → PostureState (upright / slouch / forward_head / tilted)
- Eye fatigue detection via Eye Aspect Ratio (EAR) + Mouth Aspect Ratio (MAR) — Soukupova 2016
- Emotion classification via custom CNN (4-conv-block, trained on FER2013, 7 classes: angry/disgust/fear/happy/neutral/sad/surprise)
- Vocal state classification via SVM (trained on RAVDESS, 3 classes: calm / stressed / fatigued)
- Multi-modal weighted fusion: posture 35% + fatigue 25% + emotion 25% + voice 15% = WellnessScore 0.0–1.0
- RAVDESS dataset pipeline: full audio preprocessing, 215-dim feature extraction, SVM training with GridSearchCV
- Online learning: incremental SVM retraining from opt-in user sessions

Identity layer (new):

- Face recognition: InsightFace ArcFace (512-dim embedding, cosine gallery search, FAR < 0.01%)
- Speaker identification: SpeechBrain ECAPA-TDNN (192-dim x-vector, VoxCeleb-trained)
- Identity enrollment wizard: guided 5-pose face capture + 5-sentence voice enrollment
- Identity fusion: face + voice scores combined → UserContext (CONFIRMED / FACE_ONLY / VOICE_ONLY / ANONYMOUS)
- Personalised baseline: per-user EAR / RMS / posture calibration at session start, updated via EMA

Frontend & dashboard (full replacement of Flask static):

- React 18 + TypeScript 5 + Vite 5 — modern SPA replacing existing static HTML/JS
- Node.js/Express 5 BFF (Backend for Frontend) — proxies Flask API and WebSocket to React client
- Real-time WebSocket stream: WellnessEvent pushed at ≥ 2 Hz from Python → BFF → React dashboard
- Live dashboard: personalised greeting, wellness score gauge, 4 signal cards, fusion weights bar, trend chart
- Session history page: paginated past sessions, per-session trend charts
- Nudge toasts: contextual alerts, auto-dismiss, stacked queue, nudge back-off algorithm
- Dark mode UI with signal-colour design system (sky/orange/violet/emerald per channel)

Privacy & security (full compliance stack):

- All processing on-device — no cloud inference, no cloud storage, no network required at runtime
- SQLCipher AES-256-CBC encrypted databases for face embeddings, voice voiceprints, session logs
- 4-layer consent architecture (anonymous → biometric → session log → STT transcript)
- GDPR / CCPA / BIPA compliance controls, right to erasure, data portability, retention limits
- Privacy-by-Design per Ann Cavoukian (2009) — 7 principles applied to architecture
- Model SHA-256 integrity verification at startup

Infrastructure & operations:

- pnpm monorepo: apps/web + apps/bff + packages/shared + ai_wellness_mirror (Python)
- Docker + docker-compose for local dev (facesync:local + bff + web services)
- GitHub Actions CI/CD: typecheck → lint → unit test (80% coverage gate) → integration → E2E → build → docker
- Vitest unit + integration tests; Playwright E2E tests
- Pino structured JSON logging; OpenTelemetry tracing spans on BFF routes
- Model versioning: semantic versioning, SHA-256 hashes, Git LFS, automatic rollback

1.3 Technology Stack — Complete

Layer	Technology	Version	Role
Runtime	Python	3.11+	Core AI/ML engine
Runtime	Node.js	20 LTS	BFF server
Runtime	React	18	Dashboard SPA
Build	Vite	5	Frontend bundler
Language	TypeScript	5	Type safety across BFF + frontend
Face detection	MediaPipe BlazeFace	0.10+	TFLite face detection + 478 landmarks
Face recognition	InsightFace (ArcFace)	0.7+	Face embedding + identity
Speaker ID	SpeechBrain ECAPA-TDNN	0.5+	Voice embedding + speaker identity
Emotion CNN	TensorFlow / Keras	2.15+	FER2013 trained 7-class classifier
Vocal SVM	scikit-learn	1.4+	RAVDESS trained 3-class SVM
Audio features	Librosa	0.10+	MFCC, pYIN, chroma, RMS, ZCR
Audio capture	PyAudio + sounddevice	0.2+ / 0.4+	Mic input, 16 kHz PCM
Video capture	OpenCV (cv2)	4.9+	Camera, frame processing, HUD overlay
STT	faster-whisper (int8)	1.0+	On-device English STT
BFF HTTP	Express	5	REST API proxy
BFF WebSocket	ws	8+	WebSocket proxy to React
BFF validation	Zod	3+	Runtime schema validation
BFF logging	Pino	8+	Structured JSON logs
Frontend state	TanStack Query v5	—	Server state + caching
Frontend state	Zustand	4+	UI-only client state
Frontend charts	Recharts	2+	Trend charts + score gauge
Frontend a11y	React Aria (Adobe)	—	Accessible component primitives
Database	SQLite + SQLCipher	—	Encrypted local storage
DB ORM	sqlcipher3	0.5+	Python SQLCipher binding
Web server	Flask	3.0+	Python REST + WebSocket server

Layer	Technology	Version	Role
Testing	Vitest + Playwright	—	Unit / integration / E2E
CI/CD	GitHub Actions	—	Full pipeline
Containers	Docker + docker-compose	—	Dev environment
Package mgr	pnpm workspaces	8+	Monorepo management

PART 2 — COMPLETE PROJECT STRUCTURE

2.1 Full Monorepo Directory Tree

```
FaceSync/
├── apps/
│   ├── web/                          # React 18 + TS frontend (Vite)
│   │   ├── src/
│   │   │   ├── components/
│   │   │   │   ├── ui/               # Badge, Card, Gauge, NudgeToast
│   │   │   │   ├── pipeline/         # SignalCard, FusionWeights, PipelineMap
│   │   │   │   ├── identity/         # IdentityBadge, EnrollmentWizard
│   │   │   │   └── charts/           # TrendChart, EmotionBar
│   │   │   ├── hooks/                # useWellnessStream, useSessionHistory,
useNudges
│   │   │   ├── stores/               # Zustand: uiStore
│   │   │   ├── api/                  # axios instance + React Query hooks
│   │   │   ├── utils/                # wellness helpers, formatters, a11y
│   │   │   ├── styles/               # tokens.css, reset.css, global.css
│   │   │   └── pages/
│   │   │       ├── Dashboard.tsx     # Live view
│   │   │       ├── History.tsx       # Session history
│   │   │       └── Enrollment.tsx    # Identity enrollment wizard
│   │   └── vite.config.ts            # outDir → ai_wellness_mirror/static/
│   │
│   └── bff/                          # Node.js / Express BFF
│       └── src/
│           ├── routes/
│           │   ├── wellness.ts       # GET /api/wellness/current
│           │   ├── session.ts        # GET /api/session/history
│           │   ├── identity.ts       # POST /api/identity/enroll  DELETE
/api/identity
│           │   ├── nudges.ts         # GET /api/nudges/recent
│           │   └── health.ts         # GET /health
│           ├── ws/proxy.ts           # WebSocket proxy: Flask → browser
│           ├── middleware/           # validate, logger, rateLimit, otel
│           ├── services/flaskClient.ts
│           ├── schemas/wellness.ts   # Zod schemas
│           └── app.ts                # Express app factory
│
├── packages/
│   └── shared/types/index.ts         # Shared TS types (WellnessEvent,
UserContext…)
│
├── ai_wellness_mirror/               # Python application package
│   ├── webapp.py                     # Flask REST + WebSocket server
│   ├── main.py                       # CLI entry point
│   ├── train_emotion_model.py        # CNN training script
│   │
│   ├── modules/
│   │   ├── multimodal_service.py     # Central orchestrator ★
│   │   ├── face_detection.py         # MediaPipe BlazeFace
│   │   ├── landmarks.py              # 478-point face mesh
│   │   ├── head_pose.py              # solvePnP → PostureState
│   │   ├── fatigue.py                # EAR + MAR → FatigueScore
│   │   ├── emotion.py                # CNN 7-class emotion
│   │   ├── voice.py                  # SVM 3-class vocal state
│   │   ├── fusion.py                 # Weighted signal fusion
```

```
│   │   ├── online_learning.py        # Continuous SVM retraining
│   │   ├── overlay.py                # Video HUD overlay
│   │   ├── storage.py                # CSV / SQLite session logger
│   │   └── identity/                 # NEW — identity package
│   │       ├── __init__.py
│   │       ├── face_recognizer.py    # ArcFace adapter
│   │       ├── voice_recognizer.py   # ECAPA-TDNN adapter
│   │       ├── gallery.py            # SQLCipher gallery CRUD
│   │       ├── enrollment.py         # Enrollment wizard logic
│   │       ├── profile.py            # UserProfile + baseline
│   │       └── identity_service.py   # Face + voice fusion orchestrator
│   │
│   ├── models/                       # Model weight files
│   │   ├── blaze_face_short_range.tflite
│   │   ├── face_landmarker.task
│   │   ├── emotion_model.keras
│   │   ├── arcface_r50.onnx          # NEW — ArcFace
│   │   └── ecapa_tdnn.onnx           # NEW — Speaker ID
│   │
│   ├── data/
│   │   ├── user_gallery.db           # SQLCipher — face + voice embeddings
│   │   ├── user_profiles.db          # SQLCipher — per-user baselines
│   │   └── session_logs/             # SQLCipher session logs (opt-in)
│   │
│   └── static/                       # Built React app goes here (vite outDir)
│
├── voice/
│   └── models/vocal_state_clf.pkl    # Trained SVM bundle
│
├── archive/                          # RAVDESS training dataset
├── dataset/features.npz              # Cached audio feature matrix
├── fer2013.csv                       # FER2013 emotion dataset
├── train_voice_model.py              # SVM training script
│
├── docker-compose.yml
├── Makefile
├── .env.example
├── .github/workflows/ci.yml
└── package.json                      # pnpm workspace root
```

PART 3 — 4-PHASE BUILD PLAN

3.1 Phase Overview

Phase	Name	Weeks	Goal	Acceptance Criterion
Phase 1	Core AI Engine	1–6	Build the Python AI pipeline from scratch: capture, preprocessing, all 5 models, fusion, HUD	All 5 modules producing correct output; end-to-end < 150 ms; 30-min soak test passes
Phase 2	Identity Layer	7–11	Add face recognition, speaker ID, enrollment, personalised baselines	ArcFace FAR < 0.01%; ECAPA EER < 5%; identity resolves within 3 s
Phase 3	React Frontend + BFF	12–17	Build React dashboard + Node BFF; replace Flask static UI	WebSocket stream live in browser; all Phase 1+2 signals displayed; CI/CD green
Phase 4	Privacy, Polish & Deploy	18–22	Full GDPR compliance, testing, performance hardening, production deployment	Privacy audit passes; 80% test coverage; all 10 performance targets met

PHASE 1 | CORE AI ENGINE | WEEKS 1–6

Phase 1 — Week-by-Week

Week 1: Project Scaffold & Capture

- Set up pnpm monorepo: apps/web, apps/bff, packages/shared, ai_wellness_mirror
- Create requirements.txt with all Python dependencies (pinned versions)
- Implement AudioCapture (PyAudio ring buffer, 30 ms chunks, non-blocking callback)
- Implement VideoCapture (cv2.VideoCapture, BGR→RGB, 640×480, 30 fps)
- Create CaptureThread with bounded queue.Queue(maxsize=50) for both channels
- Write 5 unit tests: chunk shape, drop count metric, queue overflow behaviour
- Set up GitHub Actions CI stub: install → typecheck → lint → test

Week 2: Face Detection & Landmarks

- Implement face_detection.py: MediaPipe BlazeFace TFLite wrapper → bounding box + confidence
- Implement landmarks.py: MediaPipe Face Landmarker → 478 (x,y,z) normalised landmarks
- Implement head_pose.py: affine alignment from 6 keypoints, OpenCV solvePnP, compute pitch/yaw/roll → PostureState enum
- Add lighting quality indicator: flag when face confidence < 0.5 for 5 consecutive frames
- Write validation tests against iBUG 300-W: landmark NME < 0.08

Week 3: Fatigue & Emotion

- Implement fatigue.py: EAR = (||p2-p6|| + ||p3-p5||) / (2 × ||p1-p4||) for both eyes; MAR for mouth; FatigueScore 0–1
- Implement emotion.py: load emotion_model.keras; preprocess face crop to 48×48 grayscale; 7-class softmax predict
- Train emotion CNN from scratch (train_emotion_model.py): 4-conv-block Keras model on FER2013; Adam lr=1e-3; EarlyStopping(patience=10); target macro-F1 ≥ 0.72

- Write unit tests: EAR formula on known landmark coords; CNN shape outputs; FER2013 test split ≥ 76% accuracy

Week 4: Voice Module & RAVDESS Pipeline

- Implement full RAVDESS dataset loader: parse Actor_XX/##-##-##-##-##-##-##.wav filename convention; extract emotion code (position 2) and intensity (position 3)
- Apply label mapping: 01,02 → calm; 05,06 → stressed; 03+intensity01 → fatigued (proxy); skip all others
- Implement AudioPreprocessor: resample → 16 kHz mono float32; trim silence (top_db=30); amplitude normalise; pad/truncate to 3 s (48,000 samples)
- Implement FeatureExtractor: 215-dim vector (MFCC×26, delta-MFCC×26, Mel×128, chroma×24, pYIN×3, RMS×2, ZCR×2, centroid×2, rolloff×2)
- Implement VoiceActivityDetector: energy-based (-40 dB threshold), spectral subtraction noise reduction
- Train SVM pipeline: StandardScaler → SVC(RBF, C=10, class_weight=balanced); GridSearchCV(cv=5, f1_macro); save pkl bundle; target macro-F1 ≥ 0.78
- Implement voice.py runtime inference: load pkl; run VAD + preprocess + feature extraction per voiced segment; emit VocalState + confidence

Week 5: Fusion, Overlay & Flask Server

- Implement fusion.py: WeightedAverage(posture=0.35, fatigue=0.25, emotion=0.25, voice=0.15) → WellnessScore 0.0–1.0; map to WellnessLevel enum
- Implement multimodal_service.py: 4-thread architecture (CaptureThread, ProcessThread, InferenceThread, FusionThread); bounded queues; typed event objects
- Implement overlay.py: OpenCV HUD — wellness score, per-signal badges, nudge text, connection status, session timer
- Implement webapp.py: Flask REST API (GET /api/wellness/current, /api/session/history, POST /api/session/start|stop) + WebSocket stream at ws://127.0.0.1:5000/ws
- Implement storage.py: CSV + SQLite session logger; opt-in gated

Week 6: Online Learning & Phase 1 Hardening

- Implement online_learning.py: background SVM retraining thread; EMA blend of new embeddings; 5-fold CV validation gate before model swap
- Implement nudge logic: score < threshold for 3 consecutive events → fire nudge; Fogg back-off (dismiss 3× → halve frequency); CRITICAL always shows
- Performance profiling: verify all p95 latency targets met (total < 200 ms); fix any bottlenecks
- 30-minute soak test: zero crashes, memory growth < 50 MB, queue depth stable
- Update CI: unit tests ≥ 80% coverage, integration tests against mock Flask

PHASE 2 | IDENTITY LAYER | WEEKS 7–11

Phase 2 — Week-by-Week

Week 7: ArcFace Face Recognition

- Add insightface, onnxruntime to requirements.txt
- Download arcface_r50.onnx (~100 MB) via scripts/download_models.py
- Implement face_recognizer.py: FaceAnalysis wrapper; aligned 112×112 crop; 512-dim L2-normalised embedding; cosine gallery search; threshold=0.40
- Implement gallery.py: SQLCipher user_gallery.db CRUD (insert embedding, fetch all, delete user, update with EMA blend α=0.1)
- Write unit tests: embedding shape (512,); known-same pair sim > 0.50; known-different pair sim < 0.20; gallery encrypt/decrypt round-trip

Week 8: ECAPA-TDNN Speaker ID

- Add speechbrain, sqlcipher3 to requirements.txt
- Download ecapa_tdnn.onnx (~80 MB) via download_models.py
- Implement voice_recognizer.py: 192-dim x-vector extraction from ≥ 1 s voiced segment; L2-normalise; cosine verification against enrolled voiceprint; threshold=0.25
- Implement noise hardening: Wiener filter when SNR < 15 dB; pre-emphasis $H(z) = 1 - 0.97z^{-1}$
- Write unit tests: embedding shape (192,); VAD gates silence correctly; speaker verification on held-out RAVDESS clips EER < 5%

Week 9: Enrollment Wizard

- Implement enrollment.py: face capture (5 guided poses); voice capture (read 5 prompted sentences ≈ 30 s); mean-pool embeddings into template; L2-normalise; verify round-trip sim > threshold; store to gallery.db
- Implement profile.py: UserProfile dataclass (user_id UUID4, display_name, face_embedding, voice_embedding, enrolled_at, session_baseline dict); stored in user_profiles.db (SQLCipher)
- Build enrollment wizard Flask route + basic HTML page (placeholder until React Enrollment.tsx is built)
- Write integration test: enroll → immediately identify → assert CONFIRMED

Week 10: Identity Service & Personalised Baselines

- Implement identity_service.py: runs at 6 fps (every 5th frame); face score + voice score fusion → UserContext(CONFIRMED / FACE_ONLY / VOICE_ONLY / ANONYMOUS / UNKNOWN); cache confirmed identity 10 s
- Implement personalised baseline calibration: 60 s resting-state capture at session start; compute EAR mean, vocal RMS mean, posture angle distribution; store as session_baseline in UserProfile
- Modify fusion.py: if UserContext.session_baseline is not None, compute deviation score instead of absolute score
- Modify multimodal_service.py: inject UserContext into InferenceThread; emit user_id in WellnessEvent

Week 11: Phase 2 Hardening & Consent System

- Implement 4-layer consent system: anonymous (no consent) → biometric (explicit) → session logging (opt-in) → STT transcript (double opt-in)
- Implement privacy controls: revoke consent; delete all data (--purge-all CLI command + Flask route); export session data (JSON)
- Implement privacy report: list all user files on device with sizes and timestamps
- Verify GDPR/CCPA compliance controls: FAR test, purge completeness test, network audit (zero bytes out)
- Phase 2 soak test: 30 min with enrolled user; identity re-verified every 10 s; zero crashes

PHASE 3 | REACT FRONTEND + NODE BFF | WEEKS 12–17

Phase 3 — Week-by-Week

Week 12: BFF Scaffold & WebSocket Proxy

- Create apps/bff package: Express 5 + TypeScript; Zod schemas mirroring all shared types
- Implement flaskClient.ts: axios instance with X-Request-ID header injection, retry logic, structured Pino logging per request
- Implement WebSocket proxy (ws/proxy.ts): bridge Flask ws://127.0.0.1:5000/ws → client /ws; ping/pong keepalive; upstream disconnect → send {type:"error",code:"FLASK_DISCONNECTED"} then close 1011
- Implement all REST routes with Zod validation: GET /api/wellness/current, /api/session/history, /api/nudges/recent, /api/identity/enroll (POST), /api/identity (DELETE), /health

- Write BFF integration tests (Vitest + supertest): /health 200; /api/wellness 503 when Flask down; invalid query params → 400 ApiError

Week 13: Design Tokens & UI Primitives

- Create tokens.css: full design system — bg/surface/border, signal colours (sky=posture, orange=fatigue, violet=emotion, emerald=voice), wellness level colours, IBM Plex Mono for data values
- Build Badge.tsx: WellnessLevel badge with icon + colour + text (never colour alone); role="status"; aria-label; CSS transition var(--ease-state)
- Build Gauge.tsx: SVG arc progress (stroke-dashoffset), aria-meter, React.memo, IBM Plex Mono score display
- Build Card.tsx: surface container, responsive padding, shadow
- Build NudgeToast.tsx: slide-in from top-right; auto-dismiss 5 s; pause on hover; role="alert"; aria-live="assertive"; max 3 stacked
- Write tests: Badge (aria-label, icon per level); Gauge (SVG arc math for 0/50/100); NudgeToast (auto-dismiss with fake timers)

Week 14: Pipeline & Chart Components

- Build SignalCard.tsx: one card per channel (posture/fatigue/emotion/voice); left-border accent in signal colour; weight shown as %; module attribution caption (e.g. "head_pose.py · solvePnP"); animated mini progress bar
- Build FusionWeights.tsx: horizontal stacked bar; segment width = weight × score; 4 signal colours; aria-label with breakdown
- Build TrendChart.tsx: Recharts AreaChart; wellness_score over time; reference lines at 0.80/0.55/0.30; custom tooltip with all signals; React.memo
- Build EmotionBar.tsx: 7-segment strip for FER2013 classes; active class highlighted in signal-emotion colour; sentiment grouping
- Build IdentityBadge.tsx: shows confirmed user name (with avatar initial) or "Anonymous" or "Unknown" with enroll prompt

Week 15: Hooks & State

- Build useWellnessStream.ts: WebSocket → BFF /ws; Zod validation on each frame; exponential back-off reconnect (1s→2s→4s→8s→30s max); ping/pong 15 s keepalive; 120-frame ring buffer (useRef); latency tracking; clean unmount
- Build useNudges.ts: merge WebSocket nudge_triggered events + poll GET /api/nudges/recent every 30 s; max 3 in queue; dismissed IDs in sessionStorage
- Build useSessionHistory.ts: TanStack Query paginated; staleTime=30 s; refetchOnWindowFocus=false
- Build uiStore.ts (Zustand): nudge queue, sidebar panel state, identity mode (enrolled/anonymous), theme preference
- Write useWellnessStream tests: connecting state; malformed frame discarded; exponential back-off timing; cleanup on unmount

Week 16: Dashboard Page & Enrollment Wizard

- Build Dashboard.tsx: CSS Grid 12-column; hero Gauge + identity badge; 4 SignalCard stack; FusionWeights bar; TrendChart (last 120 events); NudgeToast layer (position:fixed z-1000); mobile single-column at 768px breakpoint; "Reconnecting…" overlay on WS error
- Build Enrollment.tsx: guided multi-step wizard (5 face poses + voice reading); webcam preview via getUserMedia; progress indicator; consent acknowledgment with checkbox; accessible focus management between steps
- Build History.tsx: paginated SessionSummary cards; expandable TrendChart per session; date/duration/avg-score summary; CSV export button
- Build main.tsx: React ErrorBoundary; PerformanceObserver (LCP/CLS/FID to console); React QueryClient; App router

Week 17: Vite Build Integration & Phase 3 Hardening

- Configure vite.config.ts: build.outDir = ../../ai_wellness_mirror/static; emptyOutDir = true — React app builds directly into Flask static folder
- Add EnrollmentWizard routes to webapp.py and BFF to serve enrollment API
- Playwright E2E tests: Dashboard loads → WS event updates Gauge score; NudgeToast appears/dismisses; Tab navigation covers all interactive elements; 375px mobile layout correct
- Full end-to-end test: Python Flask + BFF + React all running; enroll a user; see personalised greeting in dashboard

PHASE 4 | PRIVACY, POLISH & DEPLOY | WEEKS 18–22

Phase 4 — Week-by-Week

Week 18: Full Privacy & Security Audit

- Run strace / filesystem audit: confirm zero bytes of raw video/audio written to disk at any time
- Run Wireshark capture for 30 min session: confirm zero network traffic during runtime operation
- Run SQLCipher key test: confirm database unreadable with wrong OS credential
- Run SHA-256 integrity test: confirm ModelIntegrityError fires for intentionally tampered model files
- Run GDPR compliance checklist: right to erasure (purge test), data minimisation audit, consent version hash verification, retention enforcement
- Test BIPA edge case: confirm no biometric identifier (embedding) is ever written outside SQLCipher database

Week 19: Robustness & Edge Case Testing

- Lighting: test face recognition at 500/100/30 lux; IR supplement evaluation; assert FAR < 0.1% at ≥ 50 lux
- Noise: test speaker ID at 60/30/15/10 dB SNR with babble/music/HVAC; assert EER < 5% at ≥ 15 dB SNR
- Edge cases: twins, mask/occlusion, illness voice change, bright backlight, eyeglasses, multiple people in frame, background speaker (TV), user walks away mid-session
- Demographic testing: log per-gender per-emotion accuracy; flag any gap > 5% between subgroups
- All edge case outcomes match the fallback behaviour table from Implementation Plan Section 2.5 and 3.4

Week 20: Performance Hardening

- CPU profiling (cProfile + py-spy): verify no module exceeds its allocated CPU budget; fix hotspots
- Enable progressive degradation: CPU > 80% → reduce emotion CNN rate → disable STT → reduce identity check rate → reduce landmark rate
- ONNX export of emotion CNN: convert emotion_model.keras → emotion_model.onnx; benchmark CPU inference improvement
- Enable ArcFace int8 quantisation via ONNX Runtime; benchmark latency improvement
- Final 30-min soak test on minimum-spec hardware (4-core 2.5 GHz, 6 GB RAM): all p95 latency targets met

Week 21: CI/CD & Docker

- Finalise GitHub Actions workflow: pnpm install → typecheck → lint → test:unit (80% coverage gate) → test:integration → test:e2e (Playwright headless) → build (Vite + tsc BFF) → docker build --no-cache
- Finalise docker-compose.yml: facesync (Python), bff (Node.js), web (Vite dev) services with hot-reload volumes
- Generate .env.example with all required environment variables and descriptions
- Write Makefile: make dev, make test, make build, make typecheck, make lint, make clean
- Tag v1.0.0-rc1 in Git; all model files committed to Git LFS with SHA-256 checksums.json

Week 22: Documentation, Final QA & Launch

- Write README.md: prerequisites, make dev, make test, environment variables table, architecture ASCII diagram, known limitations
- Write PRIVACY.md: complete Privacy Notice (data processed, storage location, purpose, retention, deletion, legal basis, contact)
- Write SECURITY.md: security architecture, responsible disclosure policy
- Run final full test suite: all unit + integration + E2E green; coverage ≥ 80%
- Deploy v1.0.0: tag release; publish GitHub release with model checksums; update CHANGELOG.md

PART 4 — DATASETS & TRAINING DATA

4.1 Dataset Summary

Dataset	Purpose	Size	Labels	Licence
FER2013	Train emotion CNN	35,887 images, 48×48 grayscale	7 emotion classes (angry/disgust/fear/happy/neutral/sad/surprise)	Kaggle / ICML 2013 workshop
RAVDESS	Train vocal state SVM	1,440 audio clips (WAV), 24 actors	Emotion codes 01-08 + intensity; mapped to calm/stressed/fatigued	CC BY-NC-SA 4.0
iBUG 300-W	Validate landmark accuracy	3,837 images with 68-point annotations	(x,y) landmark coordinates	Academic research (iBUG Group)
MPII Human Pose	Validate posture keypoints	~25,000 images, 16 joints per image	16 joint (x,y,visibility) tuples	Non-commercial research
LibriSpeech test-clean	Benchmark Whisper STT WER	2,620 utterances (~5.4 hrs)	Transcripts	CC BY 4.0
VoxCeleb1+2	Pre-trained ECAPA-TDNN weights	Used only for pre-training; not redistributed	Speaker identities	ECAPA model weights: Apache-2.0 via SpeechBrain

4.2 RAVDESS Label Mapping (Critical Detail)

```
# RAVDESS filename format:
# [modality]-[vocal_channel]-[emotion]-[intensity]-[statement]-[repetition]-
[actor].wav
# Emotion position: index 2 (zero-based) after splitting on "-"

EMOTION_MAP = {
    '01': 'calm',       # neutral → calm
    '02': 'calm',       # calm    → calm
    '03': None,         # happy   → fatigued IF intensity=='01' (proxy label), else
skip
    '04': None,         # sad     → skip
    '05': 'stressed',   # angry   → stressed
    '06': 'stressed',   # fearful → stressed
    '07': None,         # disgust → skip
    '08': None,         # surprised → skip
}

# Fatigued proxy rule: emotion='03' AND intensity='01' → 'fatigued'
# Document this assumption in model bundle metadata
```

⚠ **RISK / LIMITATION**

The fatigued class uses a proxy label (low-energy happy speech ≠ true vocal fatigue).
Expected fatigued class F1: ~0.72 vs. ~0.85 for calm/stressed.
Phase 2 roadmap: collect real fatigue-labelled audio from 20 opt-in pilot users.

PART 5 — AI MODEL REFERENCE

5.1 Model Summary Table

Model	File	Architecture	Input	Output	Target Accuracy
Face detector	blaze_face_short_range.tflite	MediaPipe BlazeFace TFLite (pre-trained)	Video frame RGB	Face bbox + detection confidence	> 0.8 confidence on 720p frontal face
Face landmarker	face_landmarker.task	MediaPipe Face Landmarker TFLite (pre-trained)	Cropped face	478 (x,y,z) normalised landmarks	NME < 0.08 on iBUG 300-W
Face recogniser	arcface_r50.onnx	InsightFace ArcFace ResNet-50 (pre-trained MS1MV2)	112×112 aligned face	512-dim L2-normalised embedding	FAR < 0.01% @ sim=0.40
Posture estimator	(rule-based)	OpenCV solvePnP + angle thresholds	6 facial keypoints + camera matrix	PostureState enum	≥ 85% on MPII held-out set
Fatigue detector	(formula-based)	EAR + MAR formula (Soukupova 2016)	Eye + mouth landmark coords	FatigueScore 0–1	≥ 80% accuracy on test set
Emotion CNN	emotion_model.keras	4-conv-block Keras CNN (trained from scratch on FER2013)	48×48 grayscale face crop	7-class softmax	macro-F1 ≥ 0.72
Vocal state SVM	vocal_state_clf.pkl	StandardScaler + SVC(RBF) sklearn pipeline (RAVDESS)	215-dim feature vector	VocalState + confidence	macro-F1 ≥ 0.78
Speaker ID	ecapa_tdnn.onnx	SpeechBrain ECAPA-TDNN (pre-trained VoxCeleb1+2)	16 kHz audio segment	192-dim L2-normalised x-vector	EER < 1% VoxCeleb bench
STT	Whisper tiny.en (int8)	faster-whisper CTranslate2 quantised	16 kHz audio	Transcript string	WER < 20% clean; < 35% noisy

5.2 Fusion Formula

```
# fusion.py — weighted signal fusion

WEIGHTS = { "posture": 0.35, "fatigue": 0.25, "emotion": 0.25, "voice": 0.15 }

FACE_SCORES  = { ALERT:1.0, UNKNOWN:0.7, TIRED:0.4, DISTRACTED:0.3 }
VOICE_SCORES = { CALM:1.0, NEUTRAL:0.75, FATIGUED:0.4, STRESSED:0.2 }

def compute(posture_score, fatigue_score, emotion_score, voice_score,
```

```
            user_context=None):
    # If enrolled user: use deviation from personal baseline instead of absolute
    if user_context and user_context.session_baseline:
        scores = apply_baseline_deviation(scores, user_context.session_baseline)

    composite = (WEIGHTS["posture"]  * posture_score
                 + WEIGHTS["fatigue"]  * (1 - fatigue_score)
                 + WEIGHTS["emotion"]  * emotion_score
                 + WEIGHTS["voice"]    * voice_score)

    level = ("OPTIMAL"  if composite >= 0.80 else
             "MODERATE" if composite >= 0.55 else
             "FATIGUED" if composite >= 0.30 else "CRITICAL")

    return WellnessEvent(wellness_score=composite, wellness_level=level, ...)
```

PART 6 — PERFORMANCE TARGETS & BENCHMARKS

6.1 All Performance Targets

Target	Value	Measurement Method	Phase
Posture detection accuracy	≥ 85%	MPII held-out test set	Phase 1
Eye fatigue detection accuracy	≥ 80%	Manual 50-clip annotation test	Phase 1
Emotion CNN macro-F1	≥ 0.72	FER2013 10% held-out test set	Phase 1
Vocal state SVM macro-F1	≥ 0.78	RAVDESS 20% stratified test set	Phase 1
Face recognition FAR	< 0.01%	MS-Celeb-1M benchmark; cosine sim=0.40	Phase 2
Speaker ID EER	< 1% (bench) / < 5% real-world	VoxCeleb1 / real-world 20-user test	Phase 2
End-to-end latency p50	< 130 ms	timeit 200 frames, 4-core CPU	Phase 1
End-to-end latency p95	< 200 ms	timeit 200 frames, 4-core CPU	Phase 1
Identity resolution	< 3 s first session	Stopwatch from session start	Phase 2
STT WER (clean)	< 20%	LibriSpeech test-clean 100 utterances	Phase 1
STT WER (noisy, 15 dB SNR)	< 35%	LibriSpeech + babble noise overlay	Phase 1
30-min soak — memory growth	< 50 MB	tracemalloc continuous monitoring	Phase 1+2
30-min soak — crashes	0	Automated test runner	Phase 1+2
Whisper WER after STT upgrade	< 15% (Phase 4 stretch)	LibriSpeech test-clean	Phase 4

PART 7 — PRIVACY, SECURITY & COMPLIANCE

7.1 Privacy Architecture Principles

This system applies Ann Cavoukian's 7 Privacy by Design principles (2009) as hard architectural constraints, not optional add-ons.

Principle	How It Is Applied
1. Proactive not reactive	Biometric data risks are eliminated at the design level — no raw frames ever touch disk
2. Privacy as the default	Anonymous mode is the default; enrollment is always opt-in; session logging is off by default
3. Privacy embedded into design	SQLCipher encryption, on-device processing, and ephemeral buffers are core architecture — not afterthoughts
4. Full functionality	Complete wellness functionality available in anonymous mode; enrollment adds personalisation without any trade-off
5. End-to-end security	SQLCipher AES-256; PBKDF2 key derivation; SHA-256 model integrity; no network ports exposed
6. Visibility & transparency	Privacy Report shows all data on device; Privacy Notice shown at first launch; activity indicators always visible
7. Respect for user privacy	User controls every aspect of data — retention, deletion, export, revocation — from a single Settings screen

7.2 Compliance Controls per Regulation

Regulation	Key Obligation	Implementation
GDPR Art.6	Lawful basis	Explicit informed consent before enrollment; anonymous mode requires no consent
GDPR Art.5(c)	Data minimisation	Only 512-dim embedding stored; enrollment photos discarded immediately after extraction
GDPR Art.9	Special category (biometric)	Separate explicit consent for biometric enrollment; stored encrypted; purpose-limited to wellness only
GDPR Art.17	Right to erasure	Single --purge-all command wipes all user data; confirmed by file-count check
GDPR Art.20	Data portability	Export as JSON (session labels, no embeddings) on request
GDPR Art.25	Privacy by design	See 7.1 above — all 7 principles applied
CCPA	Right to know / delete	Privacy Report + Delete All Data covers both; no third-party data sharing
BIPA (Illinois)	Biometric identifiers	No biometric identifier stored — only derived wellness labels. Embeddings treated as biometric; subject to erasure. Legal review required before IL commercial deployment
COPPA / GDPR Art.8 (minors)	Children's data	Enrollment disabled for users indicating age < 16; anonymous mode only; no session logging for minors

PART 8 — COMPLETE TESTING PLAN

8.1 Testing Pyramid

Level	Tool	Scope	Coverage Gate	When
Unit tests	Vitest (TS) + pytest (Python)	Individual functions, components, models	80% lines + 80% branches	Every PR; CI step 4
Integration tests	Vitest + supertest (BFF); pytest + requests (Python)	BFF routes; Python module interactions; database CRUD	Key paths covered	Every PR; CI step 5
E2E tests	Playwright (headless Chromium)	Full user journeys in browser	Critical paths covered	Every PR; CI step 6
Performance tests	timeit + cProfile + py-spy	Latency benchmarks, CPU profiling	All targets in Part 6 met	Weekly; pre-release
Security tests	strace + Wireshark + manual	No disk writes; no network; SQLCipher; SHA-256 integrity	100% pass required	Pre-release; Phase 4 Week 18
Soak tests	Automated 30-min test runner	Memory, crashes, queue depth, thread health	Zero crashes; < 50 MB growth	Phase 1 Week 6; Phase 2 Week 11; Phase 4 Week 20

8.2 Critical Test Cases

AI model accuracy tests:

- Emotion CNN: 3,588-image FER2013 test set; assert overall accuracy ≥ 76%; macro-F1 ≥ 0.72; disgust class F1 ≥ 0.58
- Vocal SVM: 120-clip RAVDESS test set; assert macro-F1 ≥ 0.78; fatigued class F1 ≥ 0.68
- Posture: 50 manually labelled test videos; assert ≥ 85% correct PostureState classification
- EAR fatigue: 50 annotated clips (25 tired, 25 alert); assert ≥ 80% accuracy

Identity tests:

- ArcFace: enroll 5 users × 5 images each; test recognition for each; assert all return CONFIRMED; test 5 non-enrolled users; assert all return ANONYMOUS; FAR < 1 false accept in 1,000 attempts
- ECAPA: enroll 5 users × 10 segments each; test each; assert all return CONFIRMED; test 5 non-enrolled voices; assert all ANONYMOUS; EER < 5%
- Identity fusion: assert CONFIRMED when both face + voice agree; FACE_ONLY when voice silent; ANONYMOUS when neither matches

Privacy tests:

- Filesystem audit: run full 5-min session; strace confirms 0 bytes of video/audio data written to any file outside models/ and encrypted databases
- Network audit: Wireshark captures 0 non-localhost packets during 30-min session with all features enabled
- Purge completeness: enroll user + log 3 sessions → run --purge-all → assert 0 rows in gallery, 0 rows in profiles, 0 bytes in session_logs/
- SQLCipher: open user_gallery.db with correct key → success; with wrong key → "file is not a database" error; with hex editor → binary blob, no readable embeddings

PART 9 — DEPLOYMENT & HARDWARE

9.1 Minimum & Recommended Hardware

Component	Minimum	Recommended
CPU	4-core, 2.5 GHz (Intel Core i5-8th gen or AMD Ryzen 5 3600)	6-core, 3.5 GHz+ (Intel Core i7-12th gen or AMD Ryzen 7 5800)
RAM	6 GB (4 GB base + 2 GB identity models)	8 GB+
Storage	8 GB free SSD	16 GB SSD (for model files + session data growth)
GPU	None required	NVIDIA CUDA 12.x (reduces CPU load ~40%; optional)
Camera	720p/30fps USB or built-in (UVC-compliant)	1080p/30fps USB with F/1.8 aperture + optional IR LED
Microphone	Built-in laptop mic (≥ 60 dB SNR)	USB directional condenser mic (≥ 80 dB SNR)
OS	Windows 10 / macOS 13 / Ubuntu 22.04	Windows 11 / macOS 14 / Ubuntu 24.04
Python	3.11+	3.12+
Node.js	18 LTS+	20 LTS

9.2 Installation Steps

1. git clone https://github.com/your-org/facesync && cd FaceSync
2. pip install -r requirements.txt (Python deps including all new: insightface, speechbrain, sqlcipher3, faster-whisper)
3. pnpm install (installs all Node.js deps for BFF + React frontend)
4. python scripts/download_models.py (downloads arcface_r50.onnx ~100 MB, ecapa_tdnn.onnx ~80 MB, whisper-tiny.en ~80 MB)
5. pnpm run build (builds React app → ai_wellness_mirror/static/)
6. python main.py OR docker-compose up (first-run wizard: consent → enrollment → calibration → live)

9.3 Update Strategy

Update Type	Delivery Method	Biometric Impact
Application code	Git pull + pip/pnpm install + rebuild	None — user data untouched
Model weights (breaking)	download_models.py re-run; SHA-256 verify; old model kept 7 days backup	Re-enrollment required if embedding format changed
Model weights (non-breaking)	download_models.py re-run; existing embeddings still valid	None
Privacy / consent text change	Re-consent required at next launch; old consent timestamped	None — data already stored
Security patch	Standard application update; applied automatically if auto-update consented	Patch notes displayed; re-consent only if data handling changes

PART 10 — COMPLETE BUILD CHECKLIST

10.1 Phase 1 Checklist — Core AI Engine

- AudioCapture + VideoCapture with bounded queue and drop-count metric
- CaptureThread, ProcessThread, InferenceThread, FusionThread — 4-thread model running
- face_detection.py: BlazeFace wrapper returning bounding box
- landmarks.py: 478-point face mesh returning normalised coordinates
- head_pose.py: solvePnP → PostureState enum; ≥ 85% accuracy
- fatigue.py: EAR + MAR → FatigueScore; ≥ 80% accuracy
- emotion.py: CNN 7-class; macro-F1 ≥ 0.72
- train_emotion_model.py: full training script working end-to-end
- RAVDESS loader: filename parsing with emotion/intensity mapping complete
- AudioPreprocessor: resample, trim, normalise, pad/truncate working
- FeatureExtractor: 215-dim vector with all features implemented
- VoiceActivityDetector: energy VAD + spectral subtraction working
- train_voice_model.py: SVM training + GridSearchCV + pkl bundle saved
- voice.py: runtime SVM inference working; macro-F1 ≥ 0.78
- fusion.py: weighted fusion formula; WellnessScore + WellnessLevel
- multimodal_service.py: all modules orchestrated in 4 threads
- overlay.py: HUD overlay rendering correctly
- webapp.py: Flask REST + WebSocket server working
- storage.py: session log writing with opt-in gate
- online_learning.py: background retraining with validation gate
- nudge logic: back-off algorithm working; CRITICAL override working
- 30-minute soak test passed: 0 crashes, < 50 MB memory growth
- CI pipeline: typecheck + lint + unit tests (≥ 80% coverage) all green

10.2 Phase 2 Checklist — Identity Layer

- face_recognizer.py: ArcFace adapter returning 512-dim embedding
- gallery.py: SQLCipher CRUD for embeddings working; encrypt/decrypt verified
- voice_recognizer.py: ECAPA-TDNN adapter returning 192-dim x-vector
- enrollment.py: 5-pose face + 5-sentence voice enrollment flow complete
- profile.py: UserProfile with session_baseline stored in user_profiles.db
- identity_service.py: face + voice fusion → UserContext; 10 s cache
- Personalised baseline calibration: 60 s resting-state capture at session start
- Baseline deviation scoring in fusion.py working for enrolled users
- UserContext injected into multimodal_service.py and WellnessEvent
- 4-layer consent system: all 4 layers gated correctly
- --purge-all: confirmed zero-byte purge of all user data
- Privacy Report: lists all user files with sizes and timestamps
- Phase 2 soak test: 30 min enrolled session, identity re-verified every 10 s

10.3 Phase 3 Checklist — React Frontend + Node BFF

- tokens.css: full design token system (colours, spacing, fonts, transitions)
- Badge, Gauge, Card, NudgeToast: all built with tests passing
- SignalCard, FusionWeights: all built with tests passing
- TrendChart, EmotionBar: all built
- IdentityBadge: confirms user name or anonymous status
- useWellnessStream: WS hook with reconnection, Zod validation, ring buffer
- useNudges, useSessionHistory: React Query hooks working
- Dashboard.tsx: all components integrated, mobile-responsive, WS live

- Enrollment.tsx: guided wizard; consent checkbox; face + voice capture
- History.tsx: paginated sessions, expandable trend charts, CSV export
- BFF routes: all 6 REST endpoints + WebSocket proxy working
- Vite build → ai_wellness_mirror/static/: confirmed serving from Flask
- Playwright E2E: Dashboard loads, WS updates score, NudgeToast dismisses, keyboard nav
- CI: all steps green including E2E

10.4 Phase 4 Checklist — Privacy, Polish & Deploy

- Filesystem audit: 0 bytes raw audio/video written to disk (confirmed by strace)
- Network audit: 0 non-localhost packets during session (confirmed by Wireshark)
- SQLCipher: confirmed unreadable without OS credentials
- SHA-256 integrity: ModelIntegrityError fires for tampered model
- GDPR right to erasure: purge test confirms 0 bytes user data remaining
- Demographic bias evaluation: per-subgroup accuracy breakdowns published
- All edge cases in lighting/noise/occlusion test matrix passed
- ONNX export of emotion CNN: inference latency improved
- All p95 latency targets met on minimum-spec hardware
- GitHub Actions CI: all 8 steps green consistently
- docker-compose: all 3 services start cleanly with hot-reload
- README.md, PRIVACY.md, SECURITY.md all written
- v1.0.0 tagged and released with model checksums.json

This is a living master plan. Update milestone checklist after each week. Owner: Engineering Lead.